

Lesson: Authentication, Authorization, DTOs & UI in MVC

1. The Simple Analogy: The Exclusive Airport Lounge

To understand these two concepts, imagine visiting an exclusive airport lounge:

```
[ Airport Entrance ] ---> Authentication ---> Passport / ID Check ---> Issued Boarding Pass
                                                                    |
                                                                    v
[ VIP Lounge Door ] ---> Authorization ---> Scan Boarding Pass ---> Access Granted / Denied
```

- **Authentication (Who are you?):** You present your Passport at the check-in desk. The agent verifies your face against your photo ID and hands you a Boarding Pass.
- **Authorization (What are you allowed to do?):** You walk up to the VIP Lounge. The guard scans your Boarding Pass to check if it says *First Class*.

2. System Setup (`Program.cs`)

In ASP.NET Core, cookie-based authentication is built-in. We tell the system where to send users who don't have a valid "Boarding Pass".

```
using Microsoft.AspNetCore.Authentication.Cookies;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();

// 1. Register Cookie Authentication
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";           // Where to redirect if unauthenticated
        options.AccessDeniedPath = "/Account/Denied";    // Where to redirect if unauthorized
    });

var app = builder.Build();

app.UseRouting();

// 2. Add Middleware (ORDER IS CRITICAL!)
app.UseAuthentication(); // "Who are you?" (Reads the Cookie)
app.UseAuthorization();  // "Are you allowed here?" (Checks [Authorize])

app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

3. Data Storage & DTOs (ViewModels)

We need a static class to store our users, and **ViewModels (DTOs)** to carry data securely between the UI (HTML forms) and the Controller.

The Static In-Memory Database

```
public class UserModel
{
    public string Id { get; set; } = Guid.NewGuid().ToString();
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
}

public static class InMemoryUserStore
{
    public static List<UserModel> Users = new List<UserModel>
    {
        new UserModel { Email = "admin@store.com", Password = "Password123!", FullName =
"System Admin" }
    };
}
```

The ViewModels (DTOs)

```
using System.ComponentModel.DataAnnotations;

public class LoginViewModel
{
    [Required, EmailAddress]
    public string Email { get; set; }

    [Required, DataType(DataType.Password)]
    public string Password { get; set; }
}

public class ChangePasswordViewModel
{
    [Required, DataType(DataType.Password)]
    public string CurrentPassword { get; set; }

    [Required, DataType(DataType.Password)]
    public string NewPassword { get; set; }
}

public class ForgotPasswordViewModel
{
    [Required, EmailAddress]
    public string Email { get; set; }
}
```

4. Enforcing Login: The `[Authorize]` Attribute

The VIP Bouncer

Think of `[Authorize]` as a bouncer standing in front of a Controller. If a user doesn't have an active login cookie, the bouncer catches the 401 Unauthorized error and **redirects** them to the `LoginPath` defined in `Program.cs`.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

[Authorize] // Locks down the whole controller
public class HomeController : Controller
{
    public IActionResult Index() { return View(); } // Locked

    [AllowAnonymous] // Exception: Opens this specific door to the public
    public IActionResult Contact() { return View(); } // Unlocked
}
```

5. The Core Flows: Controllers & UI (Views)

Let's look at how the DTOs, Controllers, and HTML UI work together.

Flow 1: Login

Controller Action: Validates the DTO and issues a Cookie.

```

[AllowAnonymous]
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    var user = InMemoryUserStore.Users.FirstOrDefault(u => u.Email == model.Email && u.Password == model.Password);

    if (user == null)
    {
        ModelState.AddModelError("", "Invalid email or password.");
        return View(model);
    }

    var claims = new List<Claim> {
        new Claim(ClaimTypes.Name, user.FullName),
        new Claim(ClaimTypes.Email, user.Email)
    };

    var identity = new ClaimsIdentity(claims,
        CookieAuthenticationDefaults.AuthenticationScheme);
    await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, new
        ClaimsPrincipal(identity));

    return RedirectToAction("Index", "Home");
}

```

Razor View (`Login.cshtml`): Binds the HTML form to the `LoginViewModel` .

```

@model LoginViewModel
<h2>Account Login</h2>

<form asp-action="Login" method="post">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

    <div>
        <label asp-for="Email"></label>
        <input asp-for="Email" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <div>
        <label asp-for="Password"></label>
        <input asp-for="Password" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>

    <button type="submit">Login</button>
</form>

```

Flow 2: Change Password

Controller Action: Requires an active session. Updates static memory.

```

[Authorize]
[HttpPost]
public IActionResult ChangePassword(ChangePasswordViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    string? userEmail = User.FindFirstValue(ClaimTypes.Email);
    var user = InMemoryUserStore.Users.FirstOrDefault(u => u.Email == userEmail);

    if (user == null || user.Password != model.CurrentPassword)
    {
        ModelState.AddModelError("", "Current password is incorrect.");
        return View(model);
    }

    user.Password = model.NewPassword;
    ViewBag.Message = "Password successfully changed!";
    return View();
}

```

Razor View (`ChangePassword.cshtml`):

```

@model ChangePasswordViewModel
<h2>Change Password</h2>

@if (ViewBag.Message != null) {
    <div style="color: green;">@ViewBag.Message</div>
}

<form asp-action="ChangePassword" method="post">
    <div>
        <label asp-for="CurrentPassword"></label>
        <input asp-for="CurrentPassword" />
    </div>
    <div>
        <label asp-for="NewPassword"></label>
        <input asp-for="NewPassword" />
    </div>
    <button type="submit">Update Password</button>
</form>

```

Flow 3: Forgot Password

Controller Action: Finds the user and updates the list with a temp password.

```

[AllowAnonymous]
[HttpPost]
public IActionResult ForgotPassword(ForgotPasswordViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    var user = InMemoryUserStore.Users.FirstOrDefault(u => u.Email == model.Email);
    if (user != null)
    {
        string tempPassword = "Temp" + Random.Shared.Next(1000, 9999) + "!";
        user.Password = tempPassword;
        TempData["Notice"] = $"Password reset. Temporary Password: {tempPassword}";
    }

    return RedirectToAction("ForgotPassword");
}

```

Razor View (`ForgotPassword.cshtml`):

```

@model ForgotPasswordViewModel
<h2>Forgot Password</h2>

@if (TempData["Notice"] != null) {
    <div style="color: blue;">@TempData["Notice"]</div>
}

<form asp-action="ForgotPassword" method="post">
    <div>
        <label asp-for="Email"></label>
        <input asp-for="Email" />
    </div>
    <button type="submit">Reset Password</button>
</form>

```